基于分层可导航小世界图的高效且鲁棒的近似最近邻搜索

尤·A·马尔科夫，D·A·亚舒宁

摘要— 我们提出了一种基于具有可控层次结构的可导航小世界图（Hierarchical NSW，HNSW）的近似 K-最近邻搜索新方法。所提出的解决方案完全基于图结构，无需使用额外的搜索结构——这些额外结构通常在最接近度图技术的粗略搜索阶段中使用。NSW 算法采用分层构建方式，为存储元素中的嵌套子集构建由一系列分层邻近图（层）组成的多层结构。元素所属的最大层通过指数衰减概率分布随机选取，此举可在生成与既往研究的“可导航小型世界”（NSW）结构相似的图结构的同时，确保边连接间具有特征性距离尺度分离特性。从顶层层开始搜索，并辅以尺度分离策略，相较于 NSW 算法可显著提升性能，且实现对数复杂度缩放。若进一步引入启发式算法来选择邻近图中的相邻节点，则在高召回率场景及高度聚类数据条件下性能将显著提升。性能评估表明，本文提出的通用度量空间搜索索引算法性能显著优于既往开源的顶尖纯向量搜索方法。该算法与跳表结构的相似性，使其可轻松实现均衡化的分布式部署。

索引术语— 图与树搜索策略、人工智能、信息搜索与检索、信息存储与检索、信息技术与系统、搜索过程、图与网络、数据结构、最近邻搜索、大数据、近似搜索、相似性搜索

1 引言

可用信息资源的持续增长，催生了对可扩展且高效的相似性搜索数据结构的高需求。信息搜索中常用的方法之一是 K-最近邻搜索（K- NNS）。K- NNS 假设数据元素之间存在一个定义的距离函数，其目标是从数据集中找出K个与给定查询点距离最小的元素。此类算法被广泛应用于多种场景，如非参数化机器学习算法、大规模数据库中的图像特征匹配[1]以及语义文档检索[2]。K- NNS 的一种简单方法是计算查询点与数据集中每个元素之间的距离，然后选择距离最小的元素。然而，这种简单方法的复杂度随存储元素数量呈线性增长，导致其在大规模数据集上难以实际应用。这促使人们对开发快速且可扩展的 KNNS 算法产生了浓厚兴趣。

对于 K- NNS[3-5]，精确解仅在数据维度相对较低时才能显著提升搜索速度，这归因于“维度灾难”。为克服这一问题，提出了近似最近邻搜索（K- ANNS）的概念，该概念通过允许少量近似结果，放宽了精确搜索的条件。误差。不精确搜索的质量（召回率）定义为找到的真实最近邻数量与K的比值。最常用的 K- ANNS 解决方案基于树算法的近似版本[6, 7]、局部敏感哈希（LSH）[8, 9]以及乘积量化（PQ）[10-17]。最近，邻近图 KANNS 算法[10, 18-26]也日益受到关注，在高维数据集上展现出更优的性能。然而，邻近图路由的幂律缩放特性在低维或聚类数据场景下会导致性能急剧下降。

本文提出了一种新的基于图的增量K- ANNS 结构——分层可导航小世界（Hierarchical NSW，HNSW），该结构可提供更优的对数复杂度缩放性能。主要贡献包括：显式选择图的入口节点、按不同尺度分离链接，以及采用先进的启发式算法来选择邻居节点。此外，分层可导航小世界算法可视为概率跳表结构[27]的扩展，只不过使用邻近图替代链表。性能评估表明，所提出的通用度量空间方法相较于此前仅适用于向量空间的开源前沿方法，具有显著优势。

- *Y. Malkov 任职于俄罗斯科学院应用物理研究所——联邦国家预算科研机构，地址：俄罗斯下诺夫哥罗德市 Ul ʹyanov 街 46 号，邮编 603950。电子邮箱：yurymalkov@mail.ru.*
- *D. Yashunin。地址：俄罗斯下诺夫哥罗德市 Krasnozvezdnaya 街 31-33 号，邮编 603104。电子邮箱：yashuninda@yandex.ru*

本作品已提交至 IEEE，以期发表。版权可在无需事先通知的情况下转让，转让后本版本可能将无法再访问。

2 相关工作

2.1 邻近图技术

在绝大多数已研究的图算法中，搜索过程在 k-最近邻 (k-NN) 图中均采用贪婪路由的形式[10, 18-26]。对于给定的邻近图，算法从某个起始点（可以是随机选取的，或由独立算法提供）开始搜索，并迭代地遍历该图。在遍历的每一步中，算法会计算查询点与当前基础节点的邻居节点之间的距离，然后选择使距离最小的相邻节点作为下一个基础节点，同时持续记录已发现的最佳邻居。当满足特定终止条件时（例如已完成指定次数的距离计算），搜索即告终止。k-NN 图中最近邻节点间的连接，可作为德劳内图的一种简单近似[25, 26]（该图可保证基础贪婪图遍历的结果始终为最近邻）。然而，若缺乏空间结构的先验信息，则无法高效构建德劳内图[4]；但仅利用已存储元素间的距离，即可通过最近邻节点实现其近似构建。研究表明，采用此类近似方法的邻近图算法，其性能与其他 k- ANNS 技术（如 kd-树或 LSH[18-26]）相比具有竞争力。

k-NN 图算法的主要缺陷包括：1) 在路由过程中，步骤数与数据集规模呈幂律关系[28, 29]；2) 可能导致全局连通性丧失，进而对聚类数据产生较差的搜索结果。为克服这些问题，研究者提出了多种混合算法，这些算法采用仅适用于向量数据的辅助算法（如 kd-树[18, 19]与乘积量化[10]），通过粗粒度搜索来为整节点筛选更优候选节点。

在[25, 26, 30]中，作者提出了一种称为“可导航小世界”（NSW，亦称“度量化小世界” MSW）的邻近图 K-ANNS 算法，该算法利用了*可导航*图，即在根据网络规模进行贪婪遍历时，其跳数具有对数或多项式对数量级的图[31, 32]。NSW 图是通过以随机顺序连续插入元素，并将每个新插入的元素与先前已插入元素中的M个最近邻节点双向连接而构建的。这M个最近邻节点是通过该结构的搜索过程（一种从多个随机入口节点出发的贪婪搜索变体）找到的。在构建初期插入的元素与其最近邻节点之间的连接，后来会成为网络枢纽之间的桥梁，既保持了整个图的连通性，又使得贪婪路由过程中的跳数能够呈现对数量级。

NSW 结构的构建阶段可在无需全局同步、且对精度无可量化影响的前提下高效实现并行化[26]，这使其成为分布式搜索系统的理想选择。NSW 方法在某些数据集上取得了最先进的性能[33, 34]；然而，由于整体复杂度呈多项对数级增长，该算法在低维数据集上仍易出现严重性能下降（在此类数据集上，NSW 的性能可能较基于树的算法逊色数个数量级[34]）。

2.2 可导航的小世界模型

具有贪婪图路由对数或多对数量级特性的网络被称为可导航小世界网络[31, 32]。此类网络是复杂网络理论中的重要研究课题，旨在揭示现实世界中网络形成的内在机制，进而将其应用于可扩展路由[32, 35, 36]与分布式相似性搜索[25, 26, 30, 37-40]等实际应用场景。

首批探讨可导航网络空间模型的研究由 J. Kleinberg[31, 41]完成，他将这些模型作为著名米尔格拉姆实验[42]中的社交网络模型。Kleinberg 研究了随机 Watts-Strogatz 网络的一个变体[43]，即在 d 维向量空间中使用规则格点图，并辅以遵循特定长链路长度分布r^{-q}的长链路增强。当 $q = \mathrm{d}$ 时，通过贪婪路由到达目标节点所需的跳数呈多项对数增长（而对于 q 的其他取值，则遵循幂律分布）。这一思路激发了基于导航效应开发的众多 K- NNS 和 K- ANNS 算法的发展[37-40]。尽管 Kleinberg 的可导航性准则原则上可推广至更一般的空间，但要构建此类可导航网络，仍需预先掌握数据分布信息。此外，Kleinberg 网络中的贪婪路由算法最多仅具备多项对数复杂度可扩展性。

另一类广为人知的可导航网络是无标度模型[32, 35, 36]，这类模型能够复现真实世界网络的若干特征，并已被推广用于路由应用[35]。然而，基于此类模型构建的网络在贪心搜索中的幂律复杂度缩放特性反而更为糟糕[44]；且与 Kleinberg 模型类似，无标度模型需要掌握数据分布的全局信息，这使其无法直接应用于搜索场景。

上述 NSW 算法采用了一种更简洁且此前未知的可导航网络模型，该模型支持去中心化图构建，并适用于任意空间中的数据。有研究[44]提出，NSW 网络构建机制可能是大规模生物神经网络具备可导航性的关键机制（尽管该机制是否存在尚存争议）：类似模型已成功用于描述小型脑网络的生长过程，且该模型还能预测大规模神经网络中观察到的若干高级特征。然而，NSW 模型在路由过程中仍面临多对数级搜索复杂度的问题。

3 动机

提升 NSW 搜索复杂度的方法可通过对路由过程的分析来确定，该过程已在[32, 44]中进行了详细研究。路由过程可分为两个阶段：“外扩”与“内缩”[32]。贪心算法从“外扩”阶段开始，从一个低度节点出发，在遍历图结构的同

时逐步增加该节点的度数，直至该节点连接边的特征半径达到到查询点的距离尺度。在此条件满足前，该节点的平均度数可保持相对较小，这会增加算法陷入遥远错误局部最优解的概率。

在 NSW 算法中，可通过从具有最大度数的节点开始搜索（合适的候选节点为首次插入 NSW 结构中的节点[44]），直接进入搜索的“渐进细化”阶段，从而规避上述问题。实验表明，以枢纽节点作为起点，可显著提高该结构内路由成功的概率，并在低维数据场景下带来明显性能提升。然而，该方法至多仅具备单次贪婪搜索的多对数复杂度可扩展性，且在高维数据上的性能仍逊于分层 NSW 算法。

在 NSW 算法中，单次贪婪搜索呈现多对数复杂度缩放的原因在于：总距离计算次数大致与贪婪算法的平均跳数同贪婪路径上节点的平均度数之乘积成正比。平均跳数呈对数级增长[26, 44]，而贪婪路径上节点的平均度数同样呈对数级增长，这源于以下事实：1) 随着网络规模的扩大，贪婪搜索倾向于经过相同的枢纽节点[32, 44]；2) 枢纽节点的平均连接数随网络规模增大而呈对数级增长。由此，最终复杂度呈现出整体多对数级依赖关系。

Hierarchical NSW 算法的核心思想是将链路按其长度尺度划分为不同层级，随后在多层图中进行搜索。在此框架下，对于每个元素，我们仅需独立评估所需固定数量的连接，且该过程与网络规模无关，从而实现对数级可扩展性。在此结构中，搜索从仅包含最长链路阶段（即“缩进”阶段）。该算法以贪婪方式从上层遍历各元素，直至抵达局部极小值（示意图见图1）。随后，搜索过程切换至下层（其链路长度较短），并从上一层中的局部极小值处重新启动，如此循环往复。通过将所有层中每个元素的最大连接数设为常数，可在可导航的小世界网络中实现路由算法的对数复杂度缩放。

构建此类分层结构的一种方法是：通过引入分层结构，显式地设置具有不同尺度尺度的链接。对于每个元素，我们选择一个整数层级l，该值定义了该元素所属的最大层级。对于同一层级中的所有元素，我们会逐步构建一个邻近图（即仅包含近似德劳内图的“短”链接的图）。若为层级l设定一个呈指数衰减的概率分布（即遵循几何分布），则可获得结构中预期层级数的对数尺度分布。该搜索过程采用迭代式贪心搜索策略：从顶层开始，直至零层结束。

若将来自所有层的连接进行合并，其结构将变得与 NSW 图相似（在此情况下，l可与 NSW 图中的节点度数相对应）。与 NSW 不同，分层 NSW 构建算法在插入前无需对元素进行洗牌——其随机性是通过级别随机化实现的，从而即使在数据分布暂时发生变化的情况下，仍能实现真正的增量索引（尽管由于构建过程仅具部分确定性，插入顺序的微小改变仍会影响性能）。

Hierarchical NSW 思想与一种广为人知的一维概率跳表结构[27]极为相似，且可用该结构的术语进行描述。其与跳表结构的主要区别在于：我们通过用邻近图替代链表，从而对该结构进行了泛化。Hierarchical

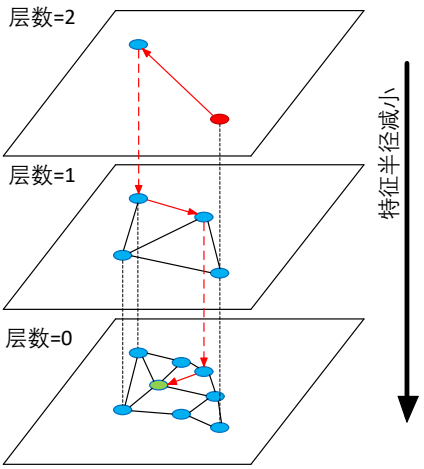


图 1. 分层 NSW 算法示意图。搜索从顶层的一个元素（以红色标示）开始。红色箭头表示从入口点到查询项（以绿色标示）的贪婪算法路径方向。

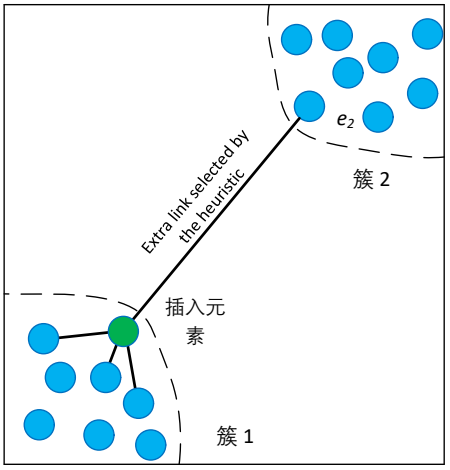


图 2. 用于为两个孤立簇选择图邻域的启发式算法示意图。一个新元素被插入到簇 1 的边界上。该元素的所有最近邻均属于簇 1，因此缺失了簇间 Delaunay 图的边。然而，该启发式算法会从簇 2 中选择元素e_2，从而在插入的新元素相较于簇 1 中的任何其他元素更接近e_2时，仍能保持全局连通性。

因此，NSW 方法可采用相同的方法来构建分布式近似搜索/叠加结构[45]。

在元素插入过程中，为选择邻近图连接，我们采用一种启发式方法：该方法综合考虑候选元素之间的距离，以构建多样化的连接（类似算法曾用于空间近似树[4]中以选择树节点子节点），而非仅选择最近邻元素。该启发式算法从距待插入元素最近的候选元素开始遍历，仅当某候选元素与基础（已插入）元素的距离小于其与已建立连接的候选元素的距离时，才与该候选元素建立连接（详见第 4 节）。

当候选点数量足够多时，该启发式算法可获取精确相对邻域图[46]作为子图——即仅利用节点间距离即可推导出的 Delaunay 图的最小子图。相对邻域图还能轻松保留全局连通分量，即使在数据高度聚集的情况下亦然（示意图见图 2）。需注意，相较于精确相对邻域图，该启发式算法会生成额外边，从而可控制连接边数，这对搜索性能至关重要。对于一维数据场景，该启发式算法仅利用元素间距离信息，即可获取精确 Delaunay 子图（此时该子图与相对邻域图完全一致），从而实现从分层 NSW 算法到一维概率跳表算法的直接转换。

分层 NSW 邻近图的基础变体也被用于文献[18]中（称为“稀疏邻域图”），用于邻近图搜索。类似的启发式方法也是 FANNG 算法[47]的研究重点。

算法 1

插入 (*hnsw*，*q*，*M*，*Mmax*，*efConstruction*，*mL)*

输入：多层图*hnsw*、新元素*q*、已建立连接数*M*、每层每个元素的最大连接数*Mmax*、动态候选列表大小*efConstruction*、层级生成归一化因子*mL*

输出：更新*hnsw*，插入元素*q*

1 *W* ← ɵ // 当前已找到的最近元素的列表
2 *ep* ← 获取*hnsw*的进入点
3 *L* ← *hnsw*的*ep* //顶层层级
4 *l* ← l-ln (*unif*(0..1)).*mL*]//新元素的层级
5**对于***lc* ←.*L l*+1
6 *W* ← 搜索层 (*q*，*ep*，*ef*=1，*lc*)
7 *ep* ← 从*Wto q*中获取最近元素
8**对于***lc* ← min(*L*，*l*) ... 0
9 *W* ← 搜索层 (*q*，*ep*，*efConstruction*，*lc)*
10*个邻居* ← select-neighbors(*q*，*W*，*M*，*lc*)//算法3 或 算法4
在层*lc*上，从 *邻居* 向*q*添加 11 条双向连接
12 **for** each $e \in neighbors$ // shrink connections if needed
13 *eConn* ←*邻域 (e)*在层*lc*
14 **if** $|eConn| > M_{max}$ // shrink connections of e
// if $l_c = 0$ then $M_{max} = M_{max0}$
15 *eNewConn* ← 选择邻近节点 (*e*，*eConn*，*Mmax*，*lc)*
//算法 3 或 算法 4
16 个集合 *邻域 (e)*在层*lc*中传输至*eNewConn*
17 *ep* ← *W*
18，**若***l*>*L*
9月19日：为*hnsw*与*q*建立连接点

（在本文手稿初版在线发布后不久发表）基于稀疏邻域图的精确路由特性[18]，提出了略有不同的诠释。

4 算法描述

网络构建算法（算法 1）通过将已存储的元素依次插入图结构来实现。对于每个被插入的元素，系统会采用指数衰减概率分布（由m_L参数归一化，参见算法 1 中的第 4 行）随机选择一个整数最大层l。

插入过程的第一阶段从顶层开始，通过贪婪地遍历图结构，以在当前层内找到与待插入元素q最接近的ef个邻近元素。随后，算法利用从前一层找到的邻近元素作为进入点，从下一层继续搜索，并重复此过程。每一层中的邻近元素均通过算法2所述的贪婪搜索算法的一个变体来寻找，该变体是[26]中算法的改进版本。为在特定层l_c中获取近似ef个最近邻元素，搜索过程中会维护一个包含ef个已发现最近邻元素的动态列表W（初始填充为进入点）。该列表在每一步中均通过评估列表中先前未评估元素的邻近区域进行更新，直至列表中所有元素的邻近区域均被评估完毕。与限制距离计算次数相比，分层 NSW 终止条件具有一项优势——它允许丢弃那些与查询点距离超过列表中最远元素之候选项，从而避免搜索结构的臃肿。与 NSW 算法类似，该算法通过两个优先队列模拟列表以提升性能。其与 NSW 算法的区别（以及部分队列优化）如下：1) 进入点为固定参数；2) 并非改变多轮搜索的次数，而是通过另一参数ef来控制搜索质量（在 NSW[26]中该参数设为K）。

算法 2

搜索层（*q*，*ep*，*ef*，*lc*）

输入：查询元素*q*、输入点*ep*、需返回距*q*最近的元素数量*ef*、层数*lc*

输出：*efclosest*至*q*的最近邻

1 *v* ← *ep* //已访问元素集合
2 *C* ← *ep* //候选集
3 *W* ← *ep* //已找到的最近邻动态列表
4**当**｜ *C* ｜> *0***时**
5 *c* ← 从*C*中提取距离*q*最近的元素
6 *f* ← 从*Wto q*获取最远元素
7**若***distance(c*，*q) > distance(f*，*q)*
8**中断**//已评估*Ware*中的所有元素
9 **for** each $e \in neighbourhood(c)$ at layer l_c // update C and W
10，**若***e* ϕ *v*
11 $v \leftarrow v \cup e$
12 *f* ← 从*Wto q*获取最远元素
13**若***distance(e*，*q) < distance(f*，*q)*或｜ *W* ｜ <*ef*
14 $C \leftarrow C \cup e$
15 $W \leftarrow W \cup e$
16 **若**｜ *W* ｜> *ef*
17 从*Wto q*中移除最远元素
18**返回***W*

算法 3
简单邻域选择 (*q*，*C*，*M*)
输入：基础元素*q*、候选元素*C*、邻近元素数量
返回*M*
输出：*M*个与*q*最接近的元素
返回*M*个距离*C*最接近*q*的元素

算法 5
K-NN 搜索（*hnsw*，*q*，*K*，*eF*）
输入：多层图*hnsw*、查询元素*q*、需返回的最近邻数量*K*、动态候选列表大小*eF*；**输出**：针对*q*返回的*K*个最近邻元素

1 *W* ← ɵ // 当前最近元素的集合
2 *ep* ← 获取*hnsw*的进入点
3 L ← *hnsw*的*ep* //顶层层级
4**对于***lc* ←*L* 1
5 *W* ← 搜索层 (*q*，*ep*，*eF*=1，*lc*)
6 *ep* ← 从*W*to *q*中获取最近元素
7 *W* ← 搜索层 (*q*，*ep*，*eF*，*lc*=0)
8**返回***K*个距离*W 到 q*最近的元素

在搜索的第一阶段，*ef*参数被设置为 1（简单贪婪搜索），以避免引入额外参数。

当搜索到达层数小于或等于*l*时，构建算法的第二阶段即被启动。该第二阶段在以下两点上有所不同：1) 将*ef*参数从1增大至*efConstruction*，以控制贪婪搜索过程的召回率；2) 将每层上找到的最近邻节点也作为待插入元素连接候选节点。

从候选节点中选取*M 个邻居*时，我们考虑了两种方法：一是与最近元素简单连接（算法3）；二是采用启发式策略——根据候选元素间的距离在多个方向建立连接（算法4），具体描述见第3节。该启发式策略包含两个附加参数：*extendCandidates*（默认设为**false**），用于扩展候选集，仅适用于高度聚集的数据；以及*keepPrunedConnections*，用于为每个元素限定固定数量的连接数。每个层中单个元素可拥有的最大连接数由参数M_{max}定义（对于大于零的层均适用此参数；基层则单独使用特殊参数M_{max0}）。若在建立新连接时某节点已达满连接状态，则其扩展连接列表将采用与邻居选择相同的算法进行缩减（算法3或4）。

当已插入元件的连接在零层上建立后，插入操作即告终止。

Hierarchical NSW 中使用的 K- ANNS 搜索算法如算法 5 所示。该算法在本质上与针对层数*l*= 0 的项的插入算法等效。不同之处在于，现在将在地面层找到的最邻近节点——这些节点被用作连接候选——作为搜索结果返回。搜索的质量由*ef*参数控制（该参数对应构建算法中的*efConstruction*）。

算法 4
邻域选择启发式算法（*q*，*C*，*M*，*lc*，*extendCandidates*，*keepPrunedConnections*）
输入参数：基础元素*q*、候选元素集合*C*、需返回的邻居数量*M*、层数*lc*、指示是否扩展候选列表的标志*extendCandidates*、指示是否添加已被剔除元素的标志*keepPrunedConnections*
输出：*M*个由启发式算法选出的元素
1 $R \leftarrow \emptyset$
2 $W \leftarrow C$ //候选者的工作队列
3 **if** *extendCandidates* // extend candidates by their neighbors
4**对于**每个*e* ϕ *C*
5**对于**每个*eadj* ϕ *第 lc 层的邻域 (e)*
6**若***eadj* ϕ *W*
7 $W \leftarrow W \cup e_{adj}$
8 Wd ← ɵ // 已丢弃候选者的队列
9**当**｜*W*｜> *0*且｜*R*｜<*M*
10 *e* ← 从*W*to *q*中提取最近元素
11**若***e*相较于*R*中的任意元素更接近*q*
12 $R \leftarrow R \cup e$
13**否则**
14 $W_d \leftarrow W_d \cup e$
15 **if** *keepPrunedConnections* // add some of the discarded
// connections from W_d
16**当**｜*Wd*｜> *0 且*｜*R*｜<*M*
17 *R* ← *R*从*Wd*中提取距离*q*最近的元素
18**返回***R*

4.1 施工参数的影响

算法构建参数m_L与M_{max0}用于维持所构建图中小世界可导航性。将m_L设为零（对应图中单层结构），并将M_{max0}设为M，可生成具有幂律搜索复杂度的有向k-NN图——该特性已在先前研究[21, 29]中得到充分验证（假设采用算法3进行邻居选择）。将m_L设为零且M_{max0}设为无穷大，则可生成具有多项式对数复杂度的NSW图[25, 26]。最后，将m_L设为非零值时，可通过引入分层结构（参见第3节），生成具有可控层级结构的图，并实现对数级搜索复杂度。

为实现可控层次结构的最佳性能优势，不同层级间相邻节点的重叠度（即属于其他层级的元素邻居所占百分比）必须保持较小。为降低该重叠度，需减小m_L；但与此同时，减小m_L会导致每层贪婪搜索过程中的平均跳跃次数增加，进而对性能产生不利影响。由此可知，m_L参数存在一个最优取值。

对于最优m_L的一个简单选择是1/ln(M)，这对应于跳表参数p=1/M，且各层间单个元素的平均重叠度为1。在Intel Core i7 5930K CPU上进行的仿真实验表明，所提出的m_L选择方案是合理的选择（关于10^4个随机d=4向量的数据参见图3）。此外，该曲线图还展示了当m_L从零开始增大时，低维数据上实现的显著加速效果，以及采用启发式方法选择图连接所产生的影响。对于高维数据则难以预期相同特性，因为此时k-NN图已具备

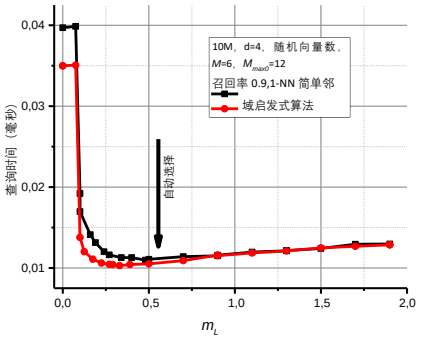


图 3. 对于 d=4 的 10M 个随机向量，查询时间与m_L参数的关系图。m_L的自动选择值 1/ln(*M*) 以箭头标示。

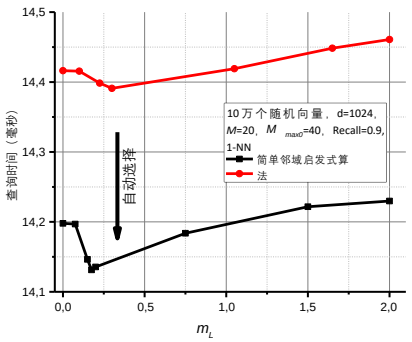


图 4. 对于 100,000 个 d=1024 的随机向量，查询时间与m_L参数的关系图。m_L的自动选择值 1/ln(*M*) 以箭头标示。

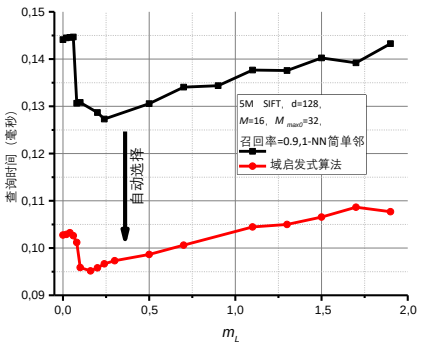


图 5. 基于 5M SIFT 学习数据集的查询时间与m_L参数关系图。针对m_L的自动选定值 1/ln(*M*) 以箭头标示。

极短的贪心算法路径[28]。令人惊讶的是，将m_L从零开始增加，在极高维数据（10万个稠密随机 d=1024 维向量，参见图 4 中的曲线）上，可显著提升速度，且不会对分层 NSW 方法造成任何性能损耗。对于如 SIFT 向量[1]（具有复杂的混合结构）等真实数据，增加m_L所带来的性能提升更为显著，但在当前设置下，相较于启发式方法带来的提升，其效果仍相对较弱（关于从 BIGANN 学习集中选取 500 万个 128 维 SIFT 向量进行 1-NN 搜索的性能结果，参见图 5[13]）。

对M_{max0}（单个元素在零层可拥有的最大连接数）的选取，也会对搜索性能产生显著影响，尤其在追求高召回率场景下。仿真实验表明，若将M_{max0}设为M（当未使用邻居选择启发式策略时，这相当于每层均采用kNN图结构），则在高召回率条件下会导致搜索性能大幅下降。仿真实验同时表明，将*Mmax0*设为2· M_{max0}是较优选择：若将该参数设得更高，则会导致性能下降及内存占用过度。图6展示了基于5M SIFT学习数据集，在不同M_{max0}参数取值下搜索性能的实验结果（实验在Intel Core i5 2400 CPU上完成）。所推荐的参数值可在不同召回率条件下均获得接近最优的性能。

在所有测试案例中，相较于直接连接最近邻节点的方法（算法3），采用基于邻近图邻域选择的启发式方法（算法4）均可获得更高或相当的搜索性能。这一优势在低维数据中最为显著，在中等维数数据中表现为高召回率，而在高度聚类数据场景中亦如此（从理论角度可将意识形态上的不连续性视为局部低维特征）——具体对比结果参见图7（Core i5 2400 CPU）。当使用最近邻节点构建邻近图时，分层NSW算法在聚类数据中难以实现高召回率，因其搜索过程易在聚类边界处陷入停滞；相反，若采用启发式方法（结合候选集扩展策略，即算法4第3行），聚类数据场景下的性能反而显著提升。对于均匀分布及极高维数据，两种邻域选择方法的性能差异较小（见图4），这可能是因为在此类场景下，几乎所有最近邻节点均能被启发式方法有效捕获。

用户唯一有意义的构建参数是*M*。*M*的合理取值范围为5至48。仿真实验表明：对于较低召回率及/或低维数据，较小的*M*通常能获得更优结果；而对于高召回率及/或高维数据，较大的*M*则更为合适（示意图参见图8，使用Core i5 2400 CPU）。该参数同时决定了算法的内存消耗（与*M*成正比），因此需谨慎选择。

*efConstruction*参数的选择十分简单。正如[26]中所建议的，该参数需足够大，以确保在构建过程中生成的 K-ANNS 召回率接近 1（对于大多数应用场景，0.95 已足够）。并且，与[26]中的做法相同，这一参数也可能

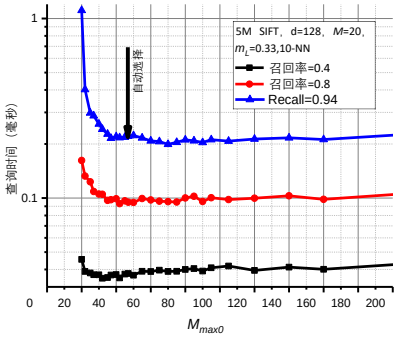


图 6。针对 5M SIFT 学习数据集，查询时间与M_{max0}参数的关系图。M_{max0}的自动选定值 2· *M*以箭头标示。

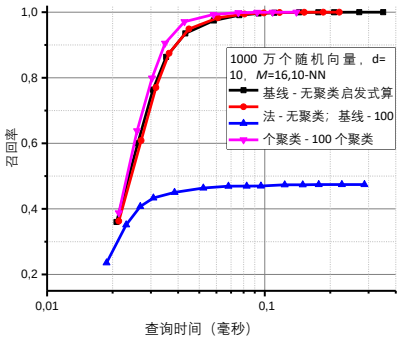


图 7. 邻域选择方法（基线对应算法 3，启发式方法对应算法 4）对聚类处理（100 个随机孤立聚类）与未聚类处理的 d=10 随机向量数据的影响。

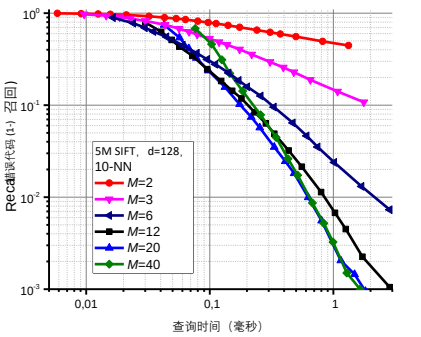


图 8. 在 5M SIFT 学习数据集上，针对 Hierarchical NSW 模型，不同*M*参数下召回率误差与查询时间的关系曲线图。

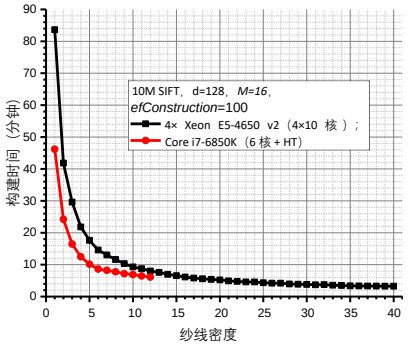


图 9. 在两个 CPU 上，针对 10M SIFT 数据集，不同线程数下 Hierarchical NSW 的构建时间。

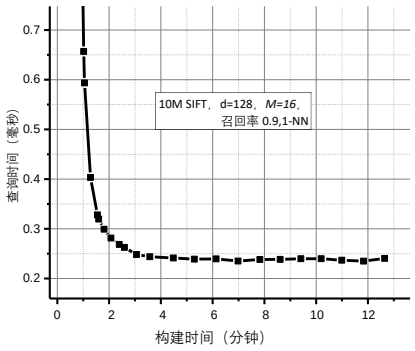


图 10. 在 10M SIFT 数据集上，Hierarchical NSW 算法的查询时间与构建时间权衡关系图。

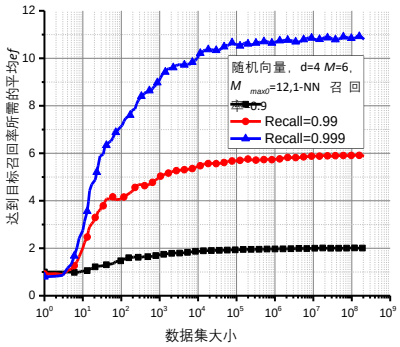


图 11. 对于 d=4 个随机向量数据，获得固定精度所需的*ef*参数与数据集规模的关系图。

可使用示例数据进行自动配置。

该构建过程仅需少量同步点（如图 9 所示），即可轻松且高效地实现并行化，且对索引质量无显著影响。构建速度与索引质量的权衡可通过*efConstruction*参数进行控制。针对 10M SIFT 数据集，图 10 展示了搜索时间与索引构建时间的权衡关系，结果表明：在配备 4×2.4 GHz、10 核 Xeon E5-4650 v2 的服务器平台上，当*efConstruction*=100 时，仅需 3 分钟即可构建出质量合格的索引。继续增大*efConstruction*值虽可带来微弱的性能提升，但代价是构建时间显著增加。

4.2 复杂度分析

4.2.1 搜索复杂度

若假设我们构建的是精确德劳内图而非近似德劳内图，则可对单次搜索的复杂度进行严格分析。假设我们已在某一层中寻得最近元素（此过程由使用德劳内图保证），随后向下遍历至下一层。可证明：在目标层内找到最近元素所需的平均步数被一个常数所限定。

事实上，各层与数据元素的空间位置无关，因此当我们遍历该图时，下一节点属于上层节点的固定概率为$p = \exp(-m_L)$。然而，在某一层内的搜索始终会在触及属于更高层的元素前终止（否则在更高层内的搜索会在不同元素上停止），故在第 s 步未能抵达目标节点的概率被上界限定为$\exp(-s \cdot m_L)$。因此，单层内所需步数的期望值被一个等比数列之和$S = 1/(1 - \exp(-m_L))$所限定，且该期望值与数据集规模无关。

若我们假设，在大规模数据集的极限情况下，Delaunay 图中每个节点的平均度数被一个常数C所限制（对于随机欧氏空间数据而言，此假设成立[48]；但在奇异空间中，该假设在原则上可能被打破），则每个层中距离计算操作的总体平均次数将由常数$C \cdot S$所限定，且该限定与数据集规模无关。

由于构建过程中最大层索引的期望值按 O(log(*N*)) 量级增长，因此整体复杂度量级为 O(log(*N*))，这与在低维数据集上进行的仿真实验结果一致。

在分层 NSW 算法中，初始假设采用精确的 Delaunay 图并不成立，因为该算法采用了基于每个元素固定数量邻居的近似边选择启发式方法。为避免陷入局部最优解，该贪心搜索算法在零层上采用了回溯机制。仿真结果表明，至少对于低维数据（图 11，d=4），为获得固定召回率所需的*ef*参数（该参数通过回溯过程中的最小跳跃次数决定算法复杂度）的依赖关系，会随数据集规模的增大而趋于饱和。回溯过程的复杂度相对于最终复杂度仅为一个加性项；因此，根据实验数据可知，Delaunay 图近似带来的误差并不会改变该复杂度缩放规律。

要对 Delaunay 图近似鲁棒性进行此类实证研究，需要获得与数据集无关的 Delaunay 图平均边数，以证明在 Hierarchical NSW 中这些边能否以恒定数量的连接被良好近似。然而，Delaunay 图的平均度数随维度呈指数级增长[39]），因此对于高维数据（例如 d=128），上述条件要求使用极其庞大的数据集，这使得此类实证研究难以实现。仍需进一步的理论分析来确认：Delaunay 图近似的鲁棒性是否可推广至更高维空间。

4.2.2 施工复杂性

该构建过程通过对所有元素进行迭代插入来完成，而单个元素的插入仅涉及在不同层上执行一系列 K-ANN 搜索，随后应用启发式方法（其在固定*efConstruction*下的复杂度为固定值）。将某个元素添加至结构所需的平均层数是一个常数，该常数取决于m_L：

$$E[l+1] = E[-\ln(unif(0,1)) \cdot m_L] + 1 = m_L + 1 \quad (1)$$

因此，插入操作的复杂度与搜索操作的复杂度相同，这意味着至少在相对低维的数据集上，构建时间复杂度为 O($N \cdot$ log(N))。

4.2.3 内存成本

Hierarchical NSW 的内存消耗主要由图连接的存储量决定。每个元素对应的连接数在零层为M_{max0}，在其他所有层中为M_{max}。因此，每个元素的平均内存消耗为（$M_{max0}+m_L \cdot M_{max}$）.bytes_per_link。若将元素总数上限设定为约40亿，则可使用4字节无符号整数来存储这些连接。测试表明，典型且接近最优的M值通常落在6至48之间。这意味着索引结构的典型内存需求（不含数据本身大小）约为每个对象60–450字节，这与仿真结果高度吻合。

5 性能评估

Hierarchical NSW 算法基于非度量空间库（nmslib）[49][1]使用 C++ 实现，而该库此前已提供一种功能完备的 NSW 实现（命名为“sw-graph”）。鉴于该库存在的若干局限性，为提升性能，Hierarchical NSW 的实现采用了自定义距离函数结合 C 风格内存管理，此举既可避免不必要的隐式寻址，又可在图遍历过程中实现高效的硬件与软件级预取。

比较 K- ANNS 算法的性能是一项非易事的任务，因为随着新算法和实现方法的不断涌现，当前最先进算法的性能也在持续变化。在本研究中，我们专注于与欧氏空间中具有开源实现的最佳算法进行比较。本文提出的分层 NSW 算法的实现也作为开源 nmslib 库[1]的一部分分发，同时提供了一个支持增量索引构建的外部 C++ 内存高效头文件版本[2]。

对比部分包含四个部分：与基准模型 NSW 的对比（5.1）；与欧氏空间中最先进算法的对比（5.2）；在 NSW 未能成功的通用度量空间中重新运行部分测试集[34]（5.3）；以及在大规模 200M SIFT 数据集上与最先进 PQ 算法的对比（5.4）。

5.1 与新南威尔士州基线值比较

在基线 NSW 算法的实现中，我们采用了来自 nmslib 1.1 的“sw-graph”（相较于[33, 34]中测试的实现略有更新），以展示其在运行速度和算法复杂度（以距离计算次数衡量）方面的改进。

图 12(a) 展示了在 Core i5-2400 CPU 上对 d=4 的随机超立方体数据集进行实验时，Hierarchical NSW 与基础 NSW 算法的性能对比结果（采用 10-NN 搜索策略）。在数据集搜索过程中，Hierarchical NSW 所需的距离计算量显著减少，尤其在高召回率场景下更为明显。

图 12(b) 展示了在 d=8 的随机超立方体数据集上，采用 10-NN 搜索且固定召回率为 0.95 时，各算法的时间复杂度缩放情况。结果清晰表明：在此实验设置下，Hierarchical NSW 的复杂度缩放特性并不劣于对数级，且在任意数据集规模下均优于 NSW。由于算法实现的优化，其在绝对运行时间方面的性能优势（图 12(c)）更为显著。

5.2 欧几里得空间中的比较

比较的主要部分是在向量数据集上进行的，使用了流行的 K- ANNS 基准测试工具 ann-benchmark [3]作为测试平台。测试系统利用算法的 Python 绑定——因此，它使用预设的算法参数，对一千个查询（从初始数据集中随机抽取）执行 K-ANN 搜索，生成包含召回率和单次搜索平均时间的输出结果。所考虑的算法包括：

1. 基于 nmslib 1.1 的 Baseline NSW 算法（“sw-graph”）。
2. FLANN 1.8.4[6]。OpenCV [5]中内置的一个常用库[4]，包含多种算法。我们使用了可用的自动调优流程，并多次重新运行，以推断出最佳参数。
3. Annoy [6]，2016年2月2日 构建。一种常用算法

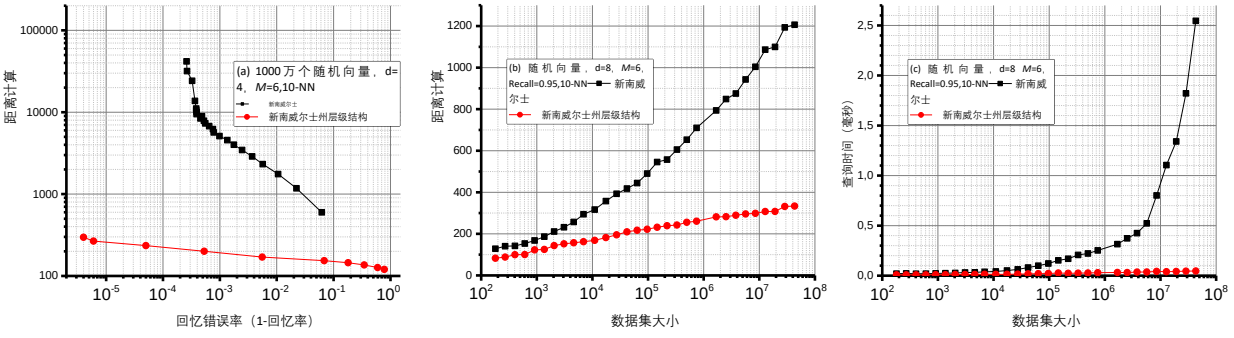


图 12. NSW 与分层 NSW 的对比：(a) 针对包含 1000 万个四维随机向量的数据集，距离计算次数与精度权衡关系；(b–c) 针对 8 维随机向量数据集，按距离计算次数 (b) 及原始查询时间 (c) 呈现的性能缩放关系。

[1] https://github.com/searchivarius/nmslib
[2] https://github.com/nmslib/hnsw
[3] https://github.com/erikbern/ann-benchmarks
[4] https://github.com/mariusmuja/flann
[5] https://github.com/opencv/opencv
[6] https://github.com/spotify/annoy

表 1

在向量空间基准测试中所用数据集的参数。

数据集	描述	尺寸	d	BF 时间	空间
筛分	图像特征向量[13]	1M	128	94 ms	L_2
GloVe	基于推文训练的词嵌入[52]	1.2M	100	95 ms	余弦
CoPhIR	从图像中提取的 MPEG-7 特征[53]	2M	272	370 ms	L_2
随机向量	超立方体中的随机向量	30M	4	590 ms	L_2
深的	从千亿级图像特征数据集[14]中抽取的一个百万子集	1M	96	60 ms	L_2
MNIST	手写数字图像[54]	60k	784	22 ms	L_2

基于随机投影树森林算法。

4. VP-tree。一种结合度量剪枝的通用度量空间算法[50]，已作为 nmslib 1.1 的一部分实现。
5. FALCONN7，版本 1.2。一种针对余弦相似度数据的新型高效 LSH 算法[51]。

本次对比实验在配备 128 GB 内存的 4X Xeon E5-4650 v2 Debian 操作系统系统上开展。针对每种算法，我们均在每个召回率区间内精心选取最佳结果，以评估在采用测试平台默认初始值时的最优性能表现。所有测试均在单线程模式下进行。Hierarchical NSW 算法则使用 GCC 5.3 编译器并启用 -Ofast 优化标志进行编译。

所用数据集的参数及描述详见表 1。除 GloVe 外，其余所有数据集均采用L_2距离；对于 GloVe，则采用余弦相似度——该指标在向量归一化后等价于L_2距离。 brute-force (BF) 时间由 nmslib 库测定。

向量数据的实验结果如图 13 所示。对于 SIFT、GloVE、DEEP 及 CoPhIR 数据集，Hierarchical NSW 的性能均显著优于其他方法。对于低维数据（d=4），Hierarchical NSW 的表现同样突出。

表 2。

用于重复“非度量数据测试”子集所用的数据集。

数据集	描述	尺寸	d	BF 时间	距离
维基稀疏	TF-IDF（词频-逆文档频率）向量（通过 GENSIM[58]生成）	4M	10^5	5.9 s	稀疏余弦
Wiki-8	基于 wiki-sparse 数据集的稀疏 TF-IDF 向量生成的主题直方图（通过 GENSIM[58]生成）	2M	8	-	詹森–香农 (JS) 散度
Wiki-128	基于 wiki-sparse 数据集的稀疏 TF-IDF 向量生成的主题直方图（通过 GENSIM[58]生成）	2M	128	1.17 s	詹森–香农 (JS) 散度
图像网	从 LSVRC -2014 中提取的签名，采用 SQFD（签名二次型）距离[59]	1M	272	18.3 s	SQFD
脱氧核糖核酸	从 Human Genome Project S[34]中采集的 DNA（脱氧核糖核酸）数据集。	1M	-	2.4 s	Levenshtein

与 Annoy 相比，其在高召回率下的速度略快，且显著优于其他算法。

5.3 一般空间中的比较

最近一项在一般空间（即非对称或违反三角不等式的空间）中对算法[34]进行的比较表明，基准算法NSW在低维数据集上存在严重问题。为了测试分层NSW算法的性能，我们重复了[34]中部分NSW表现不佳或次优的测试用例。为此，我们使用了内置的nmslib测试系统，该系统包含运行[34]中测试的脚本。评估的算法包括VP-tree、置换技术（NAPP 与暴力过滤）[49，55-57]、基础NSW算法以及由NNDescent生成的邻近图[29]（两者均与NSW图搜索算法配对使用）。与原始测试相同，对于每个数据集，测试均包含NSW或NNDescent的结果，具体取决于哪种结构表现更优。未使用自定义距离函数或特殊

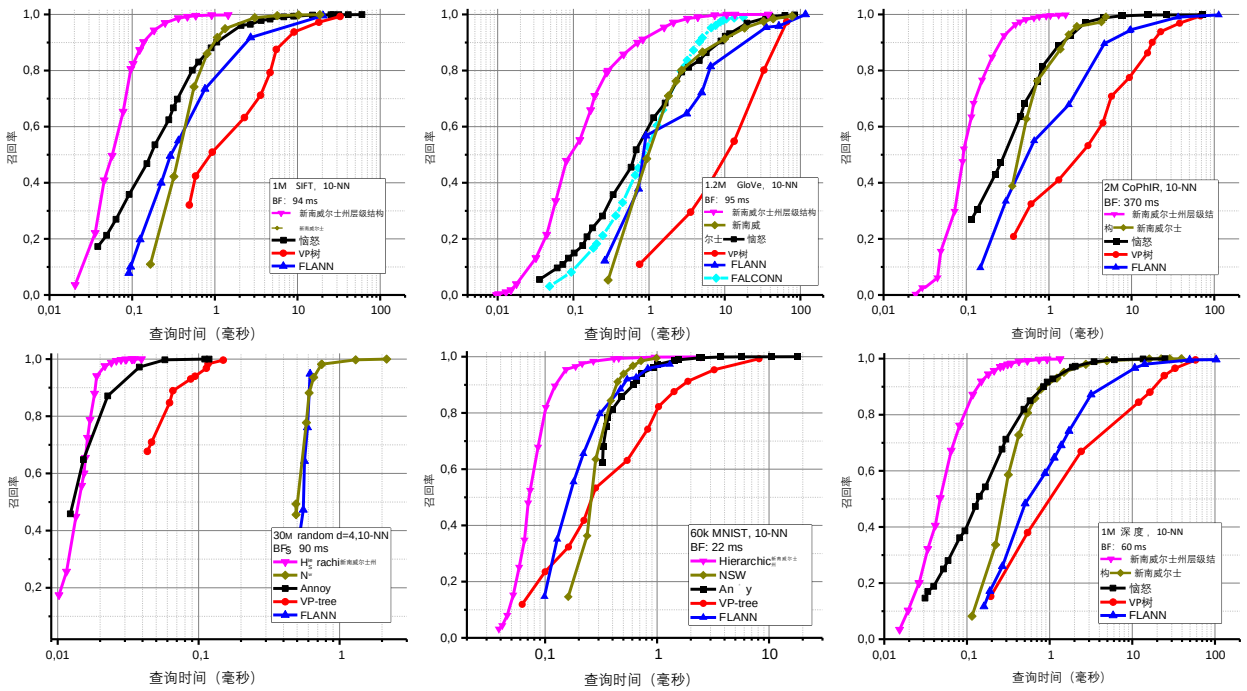


图 13. 在五个数据集上对分层 NSW 与开源 K- ANNS 算法实现结果进行比较的结果，针对 10-NN 搜索。暴力搜索的时间记为 BF。

[7] https://github.com/FALCONN-LIB/FALCONN

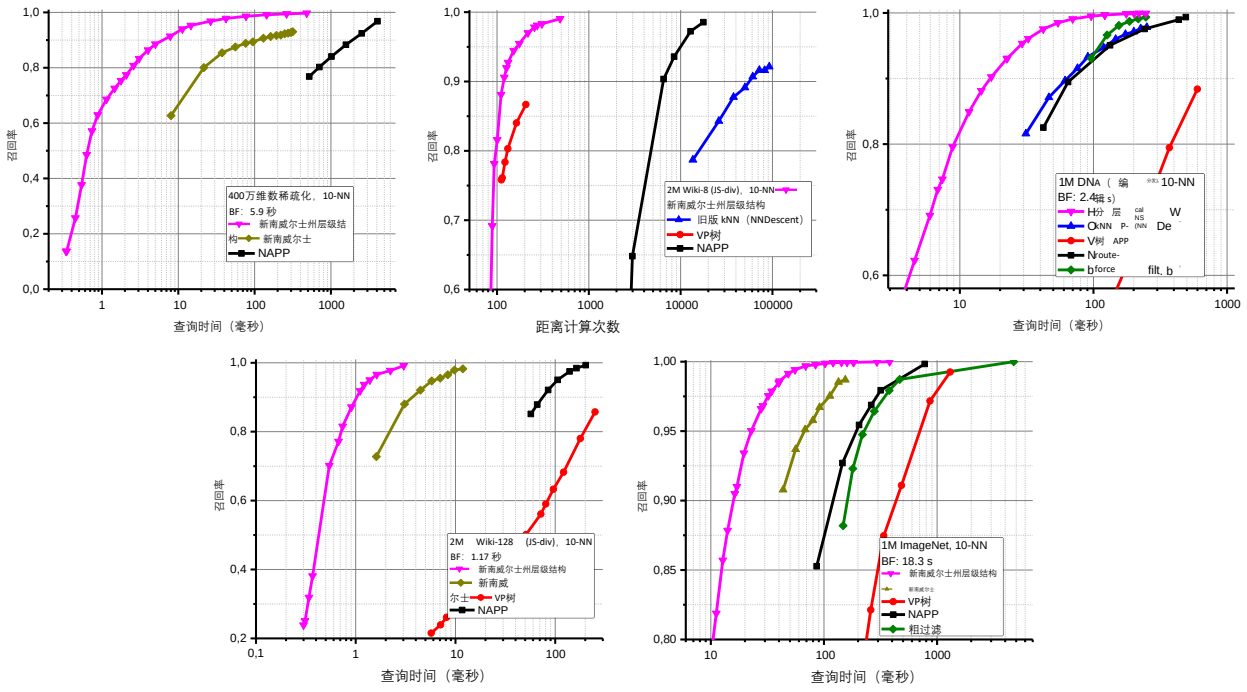


图 14. 在五个数据集上对分层 NSW 算法与来自非度量空间库的通用空间 K- ANNS 算法进行 10-NN 搜索的比较结果。蛮力搜索的时间记为 BF。

表 3。
在包含 1B 个 SIFT 样本的 200M 子集上，用于比较 Hierarchical NSW 与 Faiss 的性能参数。

算法	构建时间	峰值内存（运行时）	参数
新南威尔士州层级结构	5.6 小时	64 Gb	*M*=16, *efConstruction*=500 (1)
新南威尔士州层级结构	42 分钟	64 Gb	*M*=16, *efConstruction*=40 (2)
法伊斯	12 小时	30 Gb	*OPQ64, IMI2x14, PQ64* (1)
法伊斯	11 小时	23.5 Gb	*OPQ32, IMI2x14, PQ32* (2)

在本案例中，采用了内存管理技术以支持分层 NSW 架构，但这导致了一定的性能下降。

数据集汇总于表 2。关于数据集、空间及算法参数选择的更多详细信息，请参阅原始研究[34]。暴力搜索（BF）时间由 nmslib 库测量。

实验结果如图 14 所示。Hierarchical NSW 显著提升了 NSW 的性能，并在所有测试数据集上均表现最优。对于维度最低的数据集——采用 JS-散度评估的 wiki-8，其性能相较于 NSW 的提升幅度最大，接近三个数量级。这一重要结果表明 Hierarchical NSW 具有更强的鲁棒性，而原始 NSW 在该数据集上则面临明显瓶颈。需注意的是，对于 wiki-8 数据集，为抵消实现方式差异的影响，实验结果均以距离计算次数而非 CPU 运行时间作为基准进行呈现。

5.4 与基于产品量化的算法对比

产品量化 K- ANNS 算法[10-17]在数十亿规模的数据集上被视为最先进技术，因为它们能够高效压缩存储数据，在占用少量内存的同时，在现代 CPU 上实现毫秒级的搜索速度。

为比较 Hierarchical NSW 与 PQ 算法的性能，我们采用

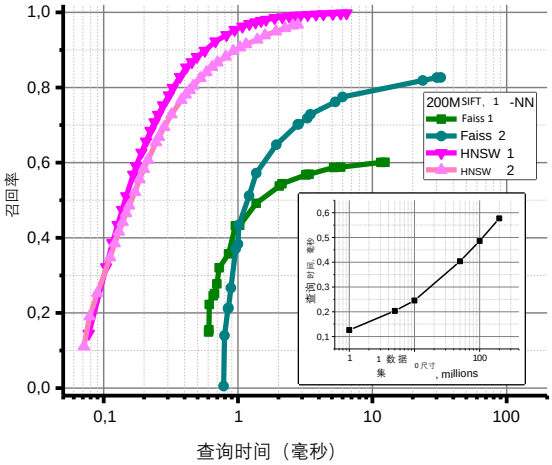


图 15 展示了在来自[13]的 200M SIFT 数据集上与 Faiss 库的对比结果。插图展示了 Hierarchical NSW 的查询时间随数据集规模变化的缩放关系。

facebook Faiss 库[8]作为基准对比库（该新库集成了最新一代 PQ 算法[12, 15]的实现，发布于本论文提交之后），并使用 OpenBLAS 后端编译。测试在包含 10 亿个 SIFT 特征的 200M 子集数据集[13]上，于配备 128GB 内存的 4X Xeon E 5-4650 v2 服务器上完成。ann-benchmark 测试平台因依赖 32 位浮点格式（仅存储数据就需要超过 100 GB 内存）而无法用于本实验。获取 Faiss PQ 算法的实验结果时，我们使用了基于 Faiss 官方文档[9]参数配置的内置脚本；而对于 Hierarchical NSW 算法，则采用了 nmslib 之外构建的专用版本——该版本内存占用较低，且采用简单非向量化实现。

[8]https://github.com/facebookresearch/faiss 2017 年 5 月构建版本。源自 2018 Faiss 库拥有其自身的 Hierarchical NSW 实现。[9]https://github.com/facebookresearch/faiss/wiki/Indexing-1G-vectors

整数距离函数以及对增量索引构建的支持[10]。

实验结果如图 15 所示，其中参数汇总见表 3。针对两种算法在构建索引后，分别使用 Linux 的 “time –v” 工具进行独立测试，以测定峰值内存占用量。尽管 Hierarchical NSW 算法所需的内存显著更多，但它仍能实现更高的精度，同时在搜索速度上取得巨大提升，且索引构建速度也大幅加快。

图 15 中的插图展示了 Hierarchical NSW 算法中查询时间随数据集规模变化的缩放关系。需要注意的是，该缩放关系与纯对数关系存在偏差，这可能是由数据集维度相对较高所致。

6 讨论

通过将可导航小世界图的结构分解与智能邻域选择启发式算法相结合，本文提出的分层 NSW 方法克服了基础 NSW 结构中的若干关键问题，从而推动了 K-ANN 搜索领域的技术前沿发展。分层 NSW 展现出卓越的性能，在多种数据集上均稳居领先地位；在高维数据场景下，其性能较开源竞品具有显著优势。即便在先前算法（NSW）性能相差数个数量级的数据集上，分层 NSW 仍能取得最优结果。此外，分层 NSW 支持连续增量索引，并可作为高效方法用于获取 k-NN 及相对邻域图的近似结果——这两者均为索引构建过程中的衍生产物。

本方法的鲁棒性是其显著优势，使其在实际应用场景中极具吸引力。该算法适用于广义度量空间，且在本文测试的所有数据集上均表现最优，从而无需针对特定问题繁琐地选择最优算法。我们特别强调该算法鲁棒性的重要性，因为数据可能具有复杂结构，且在不同尺度上呈现不同的有效维度。例如，某数据集可由位于一条曲线上、且随机填充高维立方体的点构成——此类数据在较大尺度下呈高维特性，而在较小尺度下则呈低维特性。为在此类数据集中实现高效搜索，近似最近邻算法必须在高维与低维两种情形下均具备良好性能。

还有几种方法可进一步提升Hierarchical NSW方法的效率与适用性。目前仍有一个关键参数对索引构建具有显著影响——每层新增连接数M。该参数或可通过采用不同启发式方法[4]直接推断得出。此外，若能在完整的1B SIFT与1 B DEEP数据集上对比Hierarchical NSW的性能[10-14]，并支持元素更新与删除功能，也将具有重要研究价值。

所提出的方案的一个明显缺陷是与基础 NSW 结构相比，其缺点是无法实现分布式搜索。在分层 NSW 结构中，搜索始终从顶层开始，因此由于高层元素会出现拥塞，无法采用[26]中所述的相同技术将该结构实现分布式化。可采用简单的变通方案来分布该结构，例如将数据划分至[6]中研究的集群节点上，但在此情况下，系统的总并行吞吐量并不随计算节点数量的增加而良好扩展。

尽管如此，仍存在其他已知的可行方法可将这一特定结构实现分布式部署。分层 NSW 在设计理念上与广为人知的一维精确搜索概率跳表结构高度相似，因此可采用相同技术将该结构实现分布式部署[45]。基于其对数级可扩展性及节点上理想化的均匀负载特性，相较于基础 NSW 结构，此方法有望带来更优异的分布式性能。

7 致谢

我们感谢 Leonid Boytsov 在诸多有益讨论中提供的帮助、在 Non-Metric Space Library 集成方面给予的支持，以及对本文稿提出的宝贵意见。我们感谢 Seth Hoffert 与 Azat Davletshin 对本文稿及算法提出的建议，以及在 GitHub 代码库中为算法开发做出贡献的研究人员。同时，我们感谢 Valery Kalyagin 对本研究提供的支持。

本研究报告由 RFBR 资助，项目编号为 16-31-60104 mol_a _dk。

8 参考文献

[1] D. G. Lowe，《基于尺度不变关键点提取的显著图像特征》，*《国际计算机视觉杂志》*，第 60 卷，第 2 期，第 91–110 页，2004 年。

[2] S. Deerwester、S. T. Dumais、T. K. Landauer、G. W. Furnas 及 R. A. Harshman，《基于潜在语义分析的索引构建》，*《美国信息科学学会杂志》*，第 41 卷，第 391–407 页，1990 年。

[3] P. N. Yianilos，“一般度量空间中最邻近搜索的数据结构与算法”，载于*SODA*，1993年，第93卷第194期，第311–321页。

[4] G. Navarro，《基于空间逼近的度量空间搜索》，*《The VLDB Journal》*，第 11 卷第 1 期，第 28–46 页，2002 年。

[5] E. S. Tellez、G. Ruiz 与 E. Chavez，《用于最近邻搜索的单例索引》，*《信息系统》*，2016年。

[6] M. Muja 与 D. G. Lowe，《面向高维数据的可扩展最近邻算法》，*《模式分析与机器智能》——IEEE 汇刊*，第 36 卷，第 11 期，第 2227–2240 页，2014 年。

[7] M. E. Houle 与 M. Nett，《基于秩的相似性搜索：降低维度依赖性》，*《模式分析与机器智能》——IEEE 汇刊*，第 37 卷第 1 期，第 136–150 页，2015 年。

[8] A. Andoni、P. Indyk、T. Laarhoven、I. Razenshteyn 与 L. Schmidt，《基于角度距离的实用且最优的 LSH 》，载于*《神经信息处理系统进展》*，2015年，第1225–1233页。

[9] P. Indyk 与 R. Motwani，《近似最近邻算法：迈向消除维度诅咒》，载于*《第30届 ACM 计算理论年度研讨会论文集》*，1998 年，第 604–613 页：ACM。

[10] https://github.com/nmslib/hnsw

[10] J. Wang，J. Wang，G. Zeng，R. Gan，S. Li 及 B. Guo，“基于笛卡尔拼接的快速邻域图搜索”，载于*《多媒体数据挖掘与分析》*，Springer 出版社，2015 年，第 397–417 页。

[11] M. Norouzi、A. Punjani 与 D. J. Fleet，《基于多索引哈希的汉明空间快速精确搜索》，*《模式分析与机器智能》——IEEE 汇刊*，第 36 卷第 6 期，第 1107–1119 页，2014 年。

[12] A. Babenko 与 V. Lempitsky，《逆多索引》，载于*《计算机视觉与模式识别》（CVPR），2012 年 IEEE 会议*，2012年，第 3069–3076 页：IEEE。

[13] H. Jegou、M. Douze 与 C. Schmid，《用于最近邻搜索的产物量化》，*《模式分析与机器智能》——IEEE 汇刊*，第 33 卷第 1 期，第 117–128 页，2011 年。

[14] A. Babenko 与 V. Lempitsky，《海量深度描述符数据集的高效索引》，载于*《IEEE 计算机视觉与模式识别会议论文集》*，2016 年，第 2055–2063 页。

[15] M. Douze、H. Jegou 与 F. Perronnin，《多义编码》，载于*《欧洲计算机视觉会议》*，2016年，第 785–801 页：Springer 出版。

[16] Y. Kalantidis 与 Y. Avrithis，《用于近似最近邻搜索的局部优化产物量化》，载于*《IEEE 计算机视觉与模式识别会议论文集》*，2014年，第 2321–2328 页。

[17] P. Wieschollek、O. Wang、A. Sorkine-Hornung 与 H. Lensch，“基于 GPU 的高效大规模近似最近邻搜索”，载于*《IEEE 计算机视觉与模式识别会议论文集》*，2016年，第 2027–2035 页。

[18] S. Arya 与 D. M. Mount，《固定维度下的近似最近邻查询》，载于*SODA*，1993 年，第 93 卷，第 271–280 页。

[19] J. Wang 与 S. Li，“面向大规模索引的查询驱动迭代邻域图搜索”，载于*《第 20 届 ACM 国际多媒体会议论文集》*，2012年，第 179–188 页：ACM。

[20] Z. Jiang、L. Xie、X. Deng、W. Xu 与 J. Wang，《汉明空间中的快速最近邻搜索》，载于*《多媒体建模》*，2016年，第325–336 页：Springer。

[21] E. Chavez 与 E. S. Tellez，《利用 k-最近邻图解决最近邻搜索问题》，载于*《模式识别进展》*：Springer 出版社，2010年，第 270–280 页。

[22] K. Aoyama、K. Saito、H. Sawada 与 N. Ueda，《基于度简化邻域图的快速近似相似性搜索》，载于*《第17届 ACM SIGKDD 知识发现与数据挖掘国际会议论文集》*，2011年，第1055–1063页：ACM。

[23] G. Ruiz、E. Chavez、M. Graff 及 E. S. Tellez，《通过局部搜索寻找近邻》，载于*《相似性搜索及其应用》*，Springer 出版社，2015 年，第 103–109 页。

[24] R. Paredes，《度量空间搜索中的图》，智利大学博士论文，智利，2008年。计算机科学系技术报告 TR/DCC-2008-10。可于 http://www.dcc.uchile.cl/~raparede/publ/08PhDthesis.pdf，2008 获取。

[25] Y. Malkov、A. Ponomarenko、A. Logvinov 与 V. Krylov，《高维一般度量空间中近似最近邻搜索问题的可扩展分布式算法》，载于《*相似性搜索及其应用*》：Springer 柏林-海德堡出版社，2012 年，第 132–147 页。

[26] Y. Malkov、A. Ponomarenko、A. Logvinov 及 V. Krylov，《基于可导航小世界图的近似最近邻算法》，*《信息系统》*，第 45 卷，第 61–68 页，2014 年。

[27] W. Pugh，“跳表：平衡树的概率性替代方案”，*《ACM 通讯》*，第 33 卷，第 6 期，第 668–676 页，1990 年。

[28] C. C. Cartozo 与 P. De Los Rios，《小世界网络的扩展可导航性：精确结果与新见解》，*《物理评论快报》*，第 102 卷，第 23 期，第 238703 页，2009 年。

[29] W. Dong、C. Moses 与 K. Li，“针对通用相似度度量的高效 k-最近邻图构建方法”，载于*《第 20 届万维网国际会议论文集》*，2011 年，第 577–586 页：ACM。

[30] A. Ponomarenko、Y. Malkov、A. Logvinov 与 V. Krylov，《基于小世界模型的近似最近邻搜索》，载于*《信息与通信技术及应用国际会议》*，美国佛罗里达州奥兰多，2011年。

[31] J. M. Kleinberg，《“小世界中的导航”》，*《自然》*，第 406 卷，第 6798 期，第 845–845 页，2000 年。

[32] M. Boguna、D. Krioukov 与 K. C. Claffy，《复杂网络的可导航性》，*《自然·物理》*，第 5 卷，第 1 期，第 74–80 页，2009 年。

[33] A. Ponomarenko、N. Avrelin、B. Naidan 与 L. Boytsov，《近似最近邻搜索中数据结构的对比分析》，*载于《第三届数据分析国际会议论文集》*，2014年。

[34] B. Naidan、L. Boytsov 与 E. Nyberg，《置换搜索方法高效，但仍可实现更快搜索》，*《VLDB 会议论文集》*，第 8 卷，第 12 期，第 1618–1629 页，2015 年。

[35] D. Krioukov、F. Papadopoulos、M. Kitsak、A. Vahdat 及 M. Boguna，《复杂网络的双曲几何》，*《物理评论·E》*，第 82 卷，第 3 期，第 036106 页，2010 年。

[36] A. Gulyas、J. J. Biro、A. Korosi、G. Retvari 及 D. Krioukov，《可导航网络作为导航博弈的纳什均衡》，*《自然·通讯》*，第 6 卷，第 7651 页，2015 年。

[37] Y. Lifshits 与 S. Zhang，“用于最近邻、近重复项及小世界设计的组合算法”，载于*《第20届 ACM-SIAM 离散算法年度研讨会论文集》*，2009年，第 318–326 页：工业与应用数学学会。

[38] A. Karbasi、S. Ioannidis 与 L. Massoulie，《从小世界网络到基于比较的搜索》，*《信息论：IEEE 汇刊》*，第 61 卷第 6 期，第 3056–3074 页，2015 年。

[39] O. Beaumont、A.-M. Kermarrec 与 E. Riviere，《点对点多维覆盖网络：复杂结构的近似建模》，载于*《分布式系统原理》*：Springer 出版社，2007 年，第 315–328 页。

[40] O. Beaumont、A.-M. Kermarrec、L. Marchal 与 E. Riviere，“VoroNet：基于沃罗诺伊镶嵌的可扩展对象网络”，载于*《并行与分布式处理研讨会，2007》。IPDPS 2007. IEEE 国际会议*，2007，第 1-10 页：IEEE。

[41] J. Kleinberg，《小世界现象：算法视角》，载于*《第32届ACM计算理论年度研讨会论文集》*，2000年，第163–170页：ACM。

[42] J. Travers 与 S. Milgram，《小世界问题的实验研究》，*《社会计量学》*，第 425–443 页，1969 年。

[43] D. J. Watts 与 S. H. Strogatz，《“小世界”网络的集体动力学》，*《自然》*，第 393 卷，第 6684 期，第 440–442 页，1998 年。

[44] Y. A. Malkov 与 A. Ponomarenko，《不断生长的同嗜性网络是天然的可导航小型世界》，*PLoS One*，第 e0158162 页，2016 年。

[45] M. T. Goodrich、M. J. Nelson 与 J. Z. Sun，“彩虹跳跃图：一种容错的常度分布式数据结构”，载于*《第17届 ACM-SIAM 离散算法年度研讨会论文集》*，2006年，第 384–393 页：工业与应用数学学会。

[46] G. T. Toussaint，《有限平面集合的相对邻域图》，*《模式识别》*，第 12 卷，第 4 期，第 261–268 页，1980 年。

[47] B. Harwood 与 T. Drummond，《FANNG：快速近似最近邻图》，载于*《IEEE 计算机视觉与模式识别会议论文集》*，2016年，第 5713–5722页。

[48] R. A. Dwyer，《线性期望时间下的高维 Voronoi 图》，*《离散与计算几何》*，第 6 卷，第 3 期，第 343–367 页，1991 年。

[49] L. Boytsov 与 B. Naidan，《构建高效且高效的非度量空间检索库》，载于*《相似性检索及其应用》*，Springer 出版社，2013年，第 280–293 页。

[50] L. Boytsov 与 B. Naidan，《在度量空间与非度量空间中学习剪枝》，载于*《神经信息处理系统进展》*，2013年，第 1574–1582 页。

[51] A. Andoni 与 I. Razenshteyn，《用于近似近邻搜索的最优数据依赖型哈希算法》，发表于《第 47 届 ACM 计算理论年度研讨会论文集》，美国俄勒冈州波特兰市，2015 年。

[52] J. Pennington、R. Socher 与 C. D. Manning，《Glove：词表示的全局向量》，*《自然语言处理实证方法会议论文集（EMNLP 2014）》*，第 12 卷，第 1532–1543 页，2014 年。

[53] P. Bolettieri *等人*，“CoPhIR：一种基于内容的图像检索测试集”，*arXiv 预印本 arXiv:0905.4627*，2009年。

[54] Y. LeCun、L. Bottou、Y. Bengio 与 P. Haffner，《基于梯度的学习在文档识别中的应用》，*《IEEE 会刊》*，第 86 卷第 11 期，第 2278–2324 页，1998 年。

[55] E. Chavez、M. Graff、G. Navarro 及 E. Tellez，《基于 K 个最近参考点的近邻搜索》，*《信息系统》*，第 51 卷，第 43–61 页，2015年。

[56] E. C. Gonzalez、K. Figueroa 与 G. Navarro，《通过排序置换实现有效邻近检索》，*《模式分析与机器智能》——IEEE 汇刊*，第 30 卷，第 9 期，第 1647–1658 页，2008 年。

[57] E. S. Tellez、E. Chavez 与 G. Navarro，《简明最近邻搜索》，*《信息系统》*，第 38 卷第 7 期，第 1019–1030 页，2013 年。

[58] P. Sojka，“基于大型语料库的主题建模软件框架”，载于*《LREC 2010 自然语言处理框架新挑战研讨会论文集》*，2010：Citeseer。

[59] C. Beecks，《基于距离的内容型多媒体检索相似性模型》，《亚琛莱茵-威斯特法伦应用技术大学图书馆》，2013年。



Yury A. Malkov于 2009 年获得下诺夫哥罗德国立大学物理学硕士学位，并于 2015 年获得俄罗斯科学院应用物理研究所激光物理学博士学位。他已发表 20 余篇关于物理学与计算机科学的论文。Yury 目前担任位于莫斯科的三星人工智能中心的项目负责人。他的当前研究方向包括深度学习、可扩展相似性搜索、生物神经网络及人工神经网络。



德米特里·A·亚舒宁于2009年获得下诺夫哥罗德国立大学物理学硕士学位，并于2015年获得俄罗斯科学院应用物理研究所激光物理学博士学位。2008年至2012年间，他在Mera Networks公司任职。他已发表10余篇物理学相关论文。目前，德米特里在Intelli-Vision公司担任首席研究工程师，其主要研究方向包括可扩展相似性搜索、计算机视觉及深度学习。